RELATÓRIO TÉCNICO DE PROJETO: SISTEMA EMBARCADO DE MONITORAMENTO E ALERTA MULTICANAL (WINTERHOLD)

Instituição: Pontifícia Universidade Católica de Goiás (PUC Goiás)
Curso: Engenharia de Computação
Disciplina: Sistemas Embarcados
Autores: Marini Luzia Rocha Lima, Mateus Afonso Miranda de Oliveira, Nicole de Castro Noleto, Ronaldo Oliveira de Jesus e Thiago Henrique Souza Rodrigues

1 RESUMO EXECUTIVO

O projeto Winterhold consiste em uma arquitetura distribuída de monitoramento ambiental e elétrico em tempo real voltada para infraestruturas críticas (como salas de servidores, laboratórios e ambientes climatizados). O sistema realiza a aquisição contínua de temperatura, umidade relativa do ar e tensão elétrica eficaz (RMS) por meio de nós sensores baseados no microcontrolador **ESP32**.

Diferenciando-se de arquiteturas tradicionais dependentes de servidores em nuvem ou computadores dedicados, o Winterhold consolida todo o backend de processamento, recepção de telemetria, persistência, motor de regras e servidor web diretamente em um dispositivo **Android (ARM64)** operando como Central Embarcada de Borda (Edge Gateway). O próprio pacote da aplicação (APK) executa nativamente o receptor MQTT, o servidor HTTPS autenticado, o motor de envio de alertas (via SMS nativo e SMTP TLS) e um ambiente de execução Node.js privado integrado via bibliotecas ELF para comunicação direta com a rede WhatsApp (Baileys).

2 ARQUITETURA DO SISTEMA

O sistema é dividido em três camadas operacionais distintas: **Camada de Aquisição (Nó Sensor)**, **Camada de Processamento Central (Edge Gateway Android)** e **Camada de Apresentação e Alerta (Interfaces e Canais Externos)**.

2.1 Diagrama e Responsabilidades dos Subsistemas

Subsistema / Módulo	Responsabilidade Técnica	Ambiente de Execução
Nó Sensor (Firmware)	Leitura digital do sensor DHT22 e amostragem ADC do sensor ZMPT101B; cálculo de valor eficaz (RMS); serialização JSON e publicação via protocolo MQTT.	Microcontrolador ESP32 (`esp32dev` / Framework Arduino).
Receptor de Telemetria (`MqttIngress`)	Implementação enxuta do protocolo MQTT 3.1.1 sobre TCP (porta `1883`); autenticação de credenciais, validação sintática/física do payload e encaminhamento para o núcleo.	Serviço em segundo plano no SO Android (Kotlin).
Núcleo e Motor de Regras (`DemoStore` / `DemoCentral`)	Gestão concorrente de até 32 perfis de ambientes reais; avaliação de limites inferiores/superiores; máquina de estados para deduplicação de eventos; detecção de timeout (heartbeat) e persistência atômica.	Processo principal do APK Android (`CentralService`).
Servidor Web e API (`DemoHttpServer`)	Disponibilização de ativos estáticos e API RESTful autenticada sob TLS (porta `8792`) e diagnóstico via loopback HTTP (porta `8790`).	Servidor interno em Kotlin com chaves protegidas pelo Android Keystore.
Despachante Multicanal (`AlertDelivery`)	Roteamento seletivo de alertas por severidade e variável;	Fila de trabalho dedicada no APK Android.

Subsistema / Módulo	Responsabilidade Técnica	Ambiente de Execução
	reserva atômica contra duplicidade; integração com subsistema de telefonia (`SmsManager`), cliente SMTP sobre TLS implícito (porta `465`) e ponte local de mensagens.	
Ponte de Mensagens (`EmbeddedBridge`)	Manutenção de sessão criptografada com o WhatsApp e submissão de alertas de texto mediante token de segurança.	Processo nativo Node.js 24.18.0 (`libwinterhold_node.so`) confinado em `127.0.0.1:8791`.

2.2 Fluxo de Dados de Telemetria

- **Amostragem:** O ESP32 realiza a leitura periódica das grandezas físicas e publica um objeto JSON no tópico `winterhold/<DEVICE_ID>/leituras`.
- **Ingestão e Validação:** O módulo `MqttIngress` valida as credenciais do pacote `CONNECT` e inspeciona o pacote `PUBLISH` (QoS 0). O decodificador `EspTelemetry` verifica a integridade dos tipos de dados e se os valores respeitam os limites físicos do projeto.
- **Avaliação de Regras:** A amostra recebe o carimbo de data/hora (timestamp) do relógio da central Android. Caso o `DEVICE_ID` não esteja registrado e o limite de 32 dispositivos não tenha sido atingido, o perfil é instanciado automaticamente. As variáveis não nulas são comparadas com as faixas parametrizadas.
- **Disparo e Persistência:** Havendo transição de estado (violação inédita ou normalização), gera-se um evento imutável. O despachante seleciona os destinatários ativos cujos filtros coincidam com a métrica afetada, grava a reserva em disco e aciona os canais externos. O estado consolidado é gravado automaticamente em armazenamento privado.

3 SUBSISTEMA EMBARCADO DE AQUISIÇÃO (ESP32 E SENSORES)

O nó de aquisição foi projetado sobre a plataforma **ESP32** (`esp32dev`) utilizando o ecossistema PlatformIO e bibliotecas padronizadas (DHT sensor library, Adafruit Unified Sensor e PubSubClient).

3.1 Interfaceamento de Hardware e Pinagem

- **Sensor Termo-higrométrico (DHT22):** Conectado ao pino digital **GPIO 4** (`DHT_PIN`), responsável pelas leituras de temperatura ambiente (°C) e umidade relativa (%).
- **Sensor de Tensão AC (ZMPT101B):** Conectado à entrada analógica **GPIO 34** (`ZMPT_PIN`). O conversor analógico-digital (ADC) do ESP32 opera com resolução de **12 bits** (4096 níveis) e atenuação configurada em **`ADC_11db`**, adequando a excursão do sinal analógico condicionado pelo transformador de potencial.
- **Comunicação Serial:** Interface UART configurada a **115200 baud** para depuração de conectividade e confirmação de publicação.

3.2 Processamento Digital de Sinais e Cálculo RMS

Para a medição da tensão elétrica da rede (50/60 Hz), o firmware implementa uma janela de amostragem contínua de **200 ms** por ciclo de cálculo, capturando múltiplos períodos da onda senoidal. A fim de eliminar o nível DC (offset) introduzido pelo circuito de condicionamento do módulo ZMPT101B sem depender de calibração de zero fixa, o algoritmo calcula a variância estatística das amostras em tensão (V_ADC):

- **Média:** `media = soma / quantidade_de_amostras`
- **Variância:** `variancia = (soma_dos_quadrados / quantidade_de_amostras) - media²`
- **RMS do Sinal:** `rms_do_sinal = sqrt(max(variancia, 0))`
- **Tensão Publicada:** `tensao_publicada = rms_do_sinal × ZMPT_CALIBRATION`

Por segurança operacional, o fator padrão `ZMPT_CALIBRATION` no código-fonte é inicializado em `0.0f`; caso o sistema opere sem calibração prévia (fator menor ou igual a

zero), o firmware invalida a leitura elétrica e publica explicitamente tensao_rms_v: null, evitando falsos diagnósticos.

3.3 Temporização e Resiliência de Rede no Firmware

O laço principal (`loop`) opera de forma não bloqueante respeitando as características físicas de cada transdutor:

- **Taxa de publicação MQTT:** ~1 segundo (1 Hz).
- **Taxa de amostragem do DHT22:** 2 segundos (0,5 Hz), respeitando o tempo de resposta lento do elemento sensor capacitativo/termistor. Nos ciclos intermediários de 1 segundo, a publicação preserva a última amostra termo-higrométrica válida combinada à leitura atualizada de tensão.
- **Sincronismo Temporal:** Atualização via protocolo NTP (`pool.ntp.org` e `time.google.com`).
- **Política de Reconexão:** Verificação de enlace Wi-Fi a cada 10 segundos e tentativa de reconexão ao broker MQTT a cada 5 segundos. O microcontrolador prioriza dados em tempo real, não armazenando fila em memória flash durante quedas de rede.

4 PROTOCOLO DE COMUNICAÇÃO E CONTRATO DE TELEMETRIA

Para eliminar a sobrecarga de instalar um broker externo (como Mosquitto), o APK implementa internamente o módulo `MqttIngress`, um servidor TCP concorrente dedicado ao perfil de tráfego dos microcontroladores.

- **Protocolo e Porta:** MQTT v3.1.1 sobre TCP na porta `1883`.
- **Capacidade e Limites:** Suporte a até **8 conexões simultâneas** de nós sensores, tamanho máximo de pacote de **2048 bytes** e timeout de socket de **45 segundos**.
- **Operações Suportadas:** Pacotes `CONNECT` (autenticação obrigatória com usuário `winterhold` e chave gerada pelo APK), `PUBLISH` (**QoS 0**, sem flag retain), `PINGREQ` e `DISCONNECT`.

4.2 Estrutura e Validação do Payload JSON

As mensagens devem ser endereçadas ao tópico `winterhold/<DEVICE_ID>/leituras` com tamanho entre 1 e 1024 bytes:

```
None
{
  "device_id": "sala-01",
  "timestamp": "2026-10-20T12:00:00Z",
  "temperatura_c": 24.5,
  "umidade_pct": 52.0,
  "tensao_rms_v": 220.0
}
```

A camada `EspTelemetry` aplica validação estrita de contrato antes de aceitar a amostra no estado da aplicação:

- O campo `device_id` no corpo JSON deve ser idêntico ao segmento do tópico MQTT e obedecer à expressão regular `[A-Za-z0-9_-]{1,40}`.
- Os três campos de medição (`temperatura_c`, `umidade_pct`, `tensao_rms_v`) devem estar presentes como números finitos em ponto flutuante (strings numéricas são rejeitadas) ou `null`.
- **Filtro de Plausibilidade Física:**
  - Temperatura: -40 °C a 85 °C
  - Umidade: 0% a 100%
  - Tensão: 0 V a 300 V
- Ao menos uma das três grandezas deve ser não nula; amostras integralmente nulas são descartadas e não renovam o indicador de heartbeat do dispositivo.

5 CENTRAL EMBARCADA ANDROID E ENGENHARIA DE SOFTWARE

O núcleo do sistema foi desenvolvido em **Kotlin** (SDK mínimo 26 / Android 8.0, SDK alvo 35, compilado no SDK 36 com Jetpack Compose 2026.02.01).

5.1 Serviço de Primeiro Plano e Ciclo de Vida (CentralService)

Para garantir operação ininterrupta em um sistema operacional móvel gerido por restrições severas de energia, a aplicação utiliza um serviço em primeiro plano (`foreground service` de uso especial) configurado como `START_STICKY`:

- **Retenção de Recursos:** Alocação ativa de WakeLock de CPU e WifiLock enquanto a central estiver habilitada, acompanhada de solicitação explícita de isenção de otimização de bateria (Doze Mode).

- **Laço de Supervisão:** Um escalonador interno executa ciclos de manutenção a cada **5 segundos** (avaliação de heartbeats, avanço do modo de simulação, despacho de alertas pendentes e atualização de IPs das interfaces de rede) e verificações de integridade dos subsistemas a cada **10 segundos**.
- **Recuperação Automática (`BootReceiver`):** O sistema intercepta os eventos do sistema `BOOT_COMPLETED` e `MY_PACKAGE_REPLACED`, restaurando os servidores HTTP/HTTPS, o receptor MQTT e o processo Node.js automaticamente após quedas de energia do aparelho ou atualizações de versão, respeitando o estado da flag de ativação da central.

5.2 Motor de Regras, Severidade e Deduplicação de Estado

Cada um dos 32 ambientes suportados possui limites operacionais independentes (Mínimo < Máximo). A avaliação ocorre de forma instantânea a cada amostra recebida:

- **Classificação de Severidade:**
  - Desvios térmicos ou higrométricos: **`warning`** (Aviso).
  - Desvios de tensão elétrica ou perda de comunicação (timeout > 15 segundos sem pacotes válidos): **`critical`** (Crítico).
  - Retorno da variável à faixa normal ou restabelecimento de sinal: **`info`** (Informativo, registrado diretamente como evento resolvido).
- **Máquina de Estados contra Repetição (Deduplicação):** A central rastreia o estado ativo através de uma chave composta por **`Modo (Real/Demo) + ID do Ambiente + Variável + Lado da Violação (Acima/Abaixo)`**. Se a temperatura ultrapassar o limite máximo (ex.: 28 °C para um teto de 26 °C), um evento é aberto e notificado. Amostras subsequentes ainda acima do limite (ex.: 29 °C, 27 °C) mantêm a condição ativa sem gerar inundação de mensagens (flooding). Um novo disparo para a mesma condição só é habilitado após uma leitura válida retornar efetivamente à faixa normal.
- **Separação entre Reconhecimento e Condição Física:** A ação do operador de "Marcar resolvido" atua como reconhecimento visual (acknowledge), mas preserva a trava lógica da condição física ativa (inclusive em disco), impedindo que o simples reconhecimento manual reative o envio repetido de alertas enquanto o ambiente continuar fora da faixa.

5.3 Engenharia do Runtime Integrado (Node.js + Baileys no APK)

Um dos maiores desafios técnicos do projeto foi embutir a capacidade de comunicação via WhatsApp dentro do próprio pacote Android, sem exigir aplicativos externos (como Termux) ou servidores na nuvem.

- **Empacotamento Binário (`prepare-runtime.py`):** Durante a fase de compilação, um script automatizado em Python utiliza a biblioteca **`lief` (v0.17.6)** para processar pacotes pré-compilados para Android (`arm64-v8a`), verificados por hashes SHA-256 definidos em `runtime-lock.json`. O script extrai o executável **Node.js 24.18.0** e as suas bibliotecas dinâmicas ELF dependentes, reescreve as tabelas de símbolos e caminhos de busca (`RUNPATH`/`RPATH`) para compatibilidade com o linker restrito do Android e posiciona os 10 binários resultantes em `jniLibs/arm64-v8a` (sob nomes de bibliotecas nativas como `libwinterhold_node.so`).
- **Isolamento e Execução (`EmbeddedBridge`):** Os módulos JavaScript (**Baileys 6.7.24**, Pino e QRCode) são compactados em `assets/bridge/bridge.zip` e extraídos em tempo de execução para a pasta privada `files/bridge-runtime`. As credenciais criptográficas da sessão WhatsApp ficam isoladas em `files/whatsapp-session`, preservando o pareamento entre atualizações do aplicativo.
- **Supervisão Bidirecional (Watchdog):** O processo Node escuta exclusivamente em loopback (`127.0.0.1:8791`) protegido por um token aleatório de 32+ caracteres gerado pela central. Além de o serviço Android monitorar e reiniciar o processo filho em caso de falha, o próprio script `server.js` monitora a cada 2 segundos a existência do processo pai Android (`WINTERHOLD_ANDROID_PARENT`), encerrando-se imediatamente caso fique órfão.

5.4 Subsistema de Despacho e Prevenção de Duplicidade

O envio de alertas externos percorre três canais independentes:

- **SMS Nativo:** Utiliza a API `SmsManager` do Android com divisão automática de mensagens longas (multipart) e monitoramento assíncrono via

`BroadcastReceiver` (`SmsStatusReceiver`), transitando o estado de `queued` para `submitted` mediante confirmação da operadora.

- **E-mail (SMTP sobre TLS):** Implementação própria de cliente de rede conectado via TLS implícito na porta `465`, com validação estrita de certificado/hostname, negociação `EHLO` e autenticação `AUTH LOGIN`.
- **WhatsApp:** Submissão HTTP POST local para a ponte Baileys.

Para evitar que falhas parciais de rede ou reinicializações do sistema provoquem o reenvio em massa de alertas antigos, a central implementa uma política estrita de reserva prévia (reserved) indexada pela tupla eventId + contactId + channel tanto nas preferências do Android quanto em disco na ponte Node (reserved.json). Estados terminais ou incertos (`submitted`, `failed`, `uncertain`, `blocked`) não sofrem repetição automática cega.

6 INTERFACE SINCRONIZADA, SEGURANÇA E PERSISTÊNCIA

6.1 Sincronização de Estado e Painel Web

O servidor embutido no APK disponibiliza uma Single Page Application (SPA) responsiva na porta 8792 (HTTPS) que espelha exatamente as capacidades operacionais da interface nativa em Jetpack Compose. O navegador sincroniza o estado global da central a cada **5 segundos** via `GET /api/v1/state` e aplica mutações imediatas via `POST /api/v1/action`, preservando o foco e o preenchimento de formulários em edição. Além das métricas ambientais, a API reporta em tempo real a telemetria de saúde do próprio hardware da central (percentual de bateria, uso de RAM, ocupação da partição de dados e uptime do kernel).

6.2 Mecanismos de Segurança Cibernética

- **Criptografia de Transporte (TLS):** Geração automática no aparelho de par de chaves assimétricas **EC P-256** armazenado no Android Keystore. A impressão digital **SHA-256** do certificado autoassinado é exibida na tela nativa do APK para verificação de autenticidade pelo administrador. Requisições à API feitas em texto claro (HTTP na porta `8790`) oriundas da rede local são sumariamente bloqueadas com código HTTP `403`, restringindo o HTTP exclusivamente a conexões de loopback (`127.0.0.1`) via túnel de depuração ADB.

- **Autenticação e Gestão de Sessões (`WebAccessUsers`):** A criação e revogação de contas web (até 16 usuários) é exclusiva da interface física do APK. As senhas são processadas com o algoritmo **PBKDF2-HMAC-SHA256** utilizando **210.000 iterações** e salt aleatório individual de 256 bits. As sessões autenticadas utilizam cookies com as flags `HttpOnly`, `SameSite=Strict` e `Secure`, com expiração em 8 horas.
- **Proteção contra Força Bruta e CSRF:** O endpoint `/api/v1/login` possui rate limiting em janela deslizante de 60 segundos (máximo de 5 falhas por IP e 30 globais, retornando HTTP `429`). Mutadores POST exigem o cabeçalho customizado `X-Winterhold-Request: 1` e validação estrita do cabeçalho `Origin`.
- **Proteção de Segredos em Repouso:** A senha do servidor SMTP e o token da ponte WhatsApp são cifrados com **AES-GCM** utilizando chaves simétricas geridas pelo hardware/sistema do Android Keystore.

6.3 Armazenamento e Limites de Retenção

A persistência principal (`files/demo-state.json`, esquema versão 5 com retrocompatibilidade até a versão 1) utiliza `AtomicFile` em fila assíncrona para impedir corrupção do arquivo JSON em caso de desligamento abrupto. Para garantir estabilidade de memória a longo prazo no dispositivo embarcado, aplicam-se janelas circulares de retenção:

- Até **60 amostras** retidas por ambiente (máximo de 1.920 registos reais e 300 de demonstração).
- Até **500 eventos** históricos persistidos.
- Até **100 registros de entrega** retornados por snapshot da API.

7 METODOLOGIA DE TESTES, VALIDAÇÃO E LIMITAÇÕES

7.1 Suíte de Verificação Automatizada e Integração Contínua (CI)

O projeto conta com pipelines automatizados no GitHub Actions que compilam o runtime nativo, geram os APKs e executam baterias de testes em múltiplas camadas:

- **Testes Unitários de Domínio (`DemoStoreTest` — 14 suítes):** Validam decodificação MQTT, criação e limites de perfis, isolamento entre os 5 ambientes de demonstração e os ambientes reais, transições de faixa, deduplicação de alertas, reconhecimento sem rearmamento e migração de esquemas JSON.

- **Testes da Aplicação Web (`web/test` — 8 suítes Node.js):** Verificam a lógica da interface, roteamento de destinatários, sanitização de HTML contra XSS (`escapeHtml`) e reconexão de sessão.
- **Testes Instrumentados em Emuladores Android API 30 e API 35 (`CentralIntegrationTest` e `SecurityDeliveryIntegrationTest`):** Exercitam o servidor HTTPS real com verificação de certificado, ingestão MQTT real, persistência após encerramento da `Activity`, rate-limit de login e entrega de mensagens contra receptores HTTP locais simulados.
- **Teste de Ciclo de Vida do Sistema (`test-lifecycle.py`):** Script de automação via ADB que submete o emulador à morte abrupta do processo e reinicialização completa do sistema operacional (reboot), verificando a recuperação autônoma do serviço.

7.2 Validação Experimental em Hardware Real

Os testes práticos de bancada foram conduzidos num smartphone **Samsung Galaxy A30 (`SM-A305GT`, arquitetura ARM64 `arm64-v8a`, Android 11 / API 30)** e num **Samsung Galaxy A73**:

- **Resultados Comprovados:** Instalação do pacote completo (~58,36 MiB), extração e inicialização do runtime Node.js 24.18.0 em segundo plano, geração de código QR e estabelecimento de sessão `connected` com os servidores do WhatsApp, acesso externo via LAN por HTTPS (porta `8792`), túnel de diagnóstico via ADB (porta `8790`), sobrevivência do serviço com a tela desligada, recuperação automática após término forçado do processo e reinício automático após boot do aparelho. O envio real de mensagens SMS via chip de operadora foi validado com sucesso no Galaxy A73.
- **Diagnósticos em Acompanhamento:** Durante os ensaios reais da ponte Baileys no Brasil (+55), identificou-se uma divergência no formato do identificador de conta (JID) do WhatsApp relacionada à supressão do nono dígito móvel em determinadas regiões, o que requer normalização adicional antes do envio final. Nos testes de SMTP externo, a camada de transporte TLS conectou corretamente na porta 465, exigindo apenas senhas de aplicativo (App Passwords) habilitadas no provedor de e-mail para transpor o código `535`.

7.3 Limitações de Escopo de Projeto

- **Amostragem e Histerese:** A avaliação de limites é imediata a cada amostra, sem filtro de média móvel ou banda de histerese; a janela histórica prioriza diagnóstico recente (60 amostras por dispositivo) em vez de armazenamento em banco de dados de séries temporais de longo prazo.
- **Segurança na Camada de Sensores:** O transporte MQTT na porta 1883 opera em TCP com autenticação por senha, sem camada TLS no microcontrolador, pressupondo operação em rede local (VLAN/LAN) segmentada.
- **Atuação Física:** O sistema atua estritamente em monitorização e alerta passivo/ativo, não possuindo malha de atuação física (como emissores infravermelhos para controlo de aparelhos de ar-condicionado ou relés de corte de carga).

8 CONCLUSÃO

O projeto **Winterhold** demonstra a viabilidade técnica de transformar um smartphone Android convencional em uma central de monitoramento industrial/predial completa e autossuficiente. A integração entre microcontroladores **ESP32** dedicados à aquisição sensorial (com cálculo local de tensão eficaz RMS e leitura termo-higrométrica) e um serviço nativo Android capaz de hospedar simultaneamente ingestão MQTT, interface web segura (HTTPS/TLS), persistência resiliente e um runtime Node.js nativo contorna as limitações de custo e infraestrutura de servidores dedicados, atendendo de forma robusta aos requisitos de engenharia de Sistemas Embarcados.